

Föreläsning 2

Ändrad 22 Sep 01:09

-- utskrift, variabler och heltal

Följande program skriver text på skärmen:

```
1 #include <stdio.h>
2
3 void main()
4 {
5     printf("Är jag en hacker nu, ");
6     printf("eller ...?\n");
7     printf("Va?\n");
8 }
```

De flesta program som vi ska göra under kursen följer mönstret ovan:

- `#include <stdio.h>` -- för att man ska kunna använda `printf`
- `void main()` -- talar om var *huvudprogrammet* finns, eller *huvudfunktionen* som man säger i C (*main function*)
- Därefter följer ett antal satser, mellan `{` och `}`. Det är dessa satser som är själva programmet som datorn utför
- `printf` -- skriver ut text på skärmen
- Notera citationstecknen `"` runt om texten som ska skrivas ut
- `\n` matar fram en ny rad i utskriften

Utskriften från programmet blir följande.

```
1  Är jag en hacker nu,eller ...?
2  Va?
```

Notera var det blivit radframmatningar -- jämför med utskriftssatsen! Notera att det saknas blanktecken efter kommatecknet i utskriften -- och förstå varför! (därför att vi inte bitt datorn skriva ut ett kommatecken där).

Variabler och deras typer

En variabel är som en *ask* där man kan lägga saker -- *värden* -- exempelvis ett heltal. Det får bara plats ett värde i taget.

Man måste *deklarera* de variabler man tänker använda i ett program. Det betyder att man talar om vad de heter och vilken *typ* de har. Exempel på typer är `int` och `float` som representerar *heltal* respektive *flyttal* (reella tal).

I följande program deklarerar jag sex flyttalsvariabler och två heltalsvariabler. Jag demonstrerar bland annat hur man gör för att skriva ut deras värden.

```

1  #include <stdio.h>
2
3  void main()
4  {
5      int i, j;
6      float x1, x2, y1, y2, z1, z2;
7
8      i = 10000;
9      j = 100 * 100;
10
11     x1 = 4321.0;
12     x2 = 4.321e3;
13
14     y1 = -1234;
15     y2 = -1.234e3;
16
17     z1 = 0.000000000001;
18     z2 = 1e-11;
19
20     printf("%d - %d = %d \n\n", i, j, i - j);
21
22     printf("%e - %e = %e \n", x1, x2, x1 - x2);
23     printf("%f - %f = %f \n", y1, y2, y1 - y2);
24     printf("%g - %g = %g \n", z1, z2, z1 - z2);
25 }

```

Notera de många olika sätten att specificera vad som ska skrivas ut: `%d`, `%e`, `%f` och `%g`. Såhär ser utskriften ut:

```

1  10000 - 10000 = 0
2
3  4.321000e+03 - 4.321000e+03 = 0.000000e+00
4  -1234.000000 - -1234.000000 = 0.000000
5  1e-11 - 1e-11 = 0

```

Alla differenser blir noll. Trots att tilldelningssatserna såg olika ut får variablerna parvis samma värden. Om du tittar på tilldelningsatserna (raderna 11--18 så ser du reglerna för hur man skriver flyttal i program. Tecknet `e` är till för tiopotensnotation. Exempel från rad 15:

$$1.234e3 = 1234 \times 10^3$$

Följande gäller för flyttalstypen `float` i `printf`:

- `%e` ger exponentform med sex decimaler
- `%f` ger "vanlig" form, sex decimaler
- `%g` är "intelligent". Datorn väljer lämplig form.
- Det finns fler möjligheter. Se kap 3.9 i [kursboken](#).

Inom parentes Datorn omvandlar talen mellan text och sin interna representation av flyttal. Detta händer både vid tilldelning och utskrift. Datorns interna representation är en exponentform. Men den är inte decimal utan binär, som allt annat i datorn. Vi återkommer till detta strax.

Heltalstyper och deras gränser

Matematikerna har bara *en* sorts heltal. C-programmeraren har tillgång till inte mindre än åtta olika sorters heltal -- åtta olika heltalstyper. Jag ska genom ett exempel demonstrera vad det är för skillnad på dessa heltalstyper.

I följande program använder jag tre heltalstyper: `short`

`int`, `int` och `long int`.

```

1  #include <stdio.h>
2
3  void main()
4  {
5      long int days;
6      int minutes;
7      short int seconds;
8
9      printf("Ett år \n");
10
11     days = 365;
12     printf("= %ld dagar\n", days);
13
14     minutes = days * 24 * 60;
15     printf("= %d minuter\n", minutes);
16
17     seconds = minutes * 60;
18     printf("= %d sekunder\n", seconds);
19 }
```

Jag deklarerar tre heltalsvariabler: `days`, `minutes` och `seconds`. Sedan använder jag dem till att räkna ut hur många dagar, minuter och sekunder som går på ett år. Variablerna får sina värden i tilldelningssatser (rad 11, 14 och 17).

Utskriften blir följande (`long int` skrivs ut med specifikationen `%ld`):

```

1  Ett år
2  = 365 dagar
3  = 525600 minuter
4  = 13184 sekunder
```

Och det stämmer inte! Dagar och minuter stämmer. Men sekunderna är fel. Det ska vara *3153600* sekunder.

Nu ska jag förklara varför det blir fel.

Det som skiljer heltalstyperna åt är hur stora de kan bli. Följande gäller på det datorsystem där jag skriver mina C-program:

```

■ -32768 <= short int <= 32767
■ -2147483648 <= long int <= 2147483647
■ int = long int
```

Märkliga intervall kan det synas. Men det har att göra med att datorn räknar i det binära talsystemet. Uttryckt i binära, oktala eller hexadecimala tal, ser intervallen rimligare ut, exempelvis `short int`:

```

■ -0x8000 <= short int <= 0x7fff
■ -0100000 <= short int <= 077777
■ -1000 0000 0000 0000 <= short int <= 0111 1111
1111 1111
```

Inom parentes Man kan skriva konstanter oktalt och hexadecimalt i C. Man inleder -- precis som ovan -- med `0x` för hexadecimalt, och med bara `0` (en nolla) för oktalt. (Vid inläsning med `scanf` gäller dock andra regler).

Intervallen för heltalstyperna behöver *inte* vara som ovan på det system där ni jobbar. Typerna `long int` och `short int` är dock garanterat *åtminstone* så stora som på mitt system.

Åter till den felaktiga sekundberäkningen!

Det som händer är att beräkningen ramlar utanför det intervall som är tillåtet för en `short int`. Om vi deklarerar `seconds` till en `int` eller en `long int` så får vi det rätta svaret.

OBS! Vi får ingen varning när sådana här saker händer!

! Datorn låtsas inte alls om att han hamnat utanför det tillåtna intervallet. Han räknar vidare med en egen inre logik -- han snurrar runt bland de tillåtna värdena. !

Det är upp till programmeraren att se till att beräkningarna stannar inom tillåtna intervall!

Låt mig slutligen nämna samtliga heltalstyper, med de intervall som gäller på mitt system. Namnen på heltalstyperna kan ofta förkortas. Exempelvis skriver man oftast bara `short` och `long` om `short int` respektive `long int`.

```

1      -128 <=    signed char    <=      127
2          0 <=    unsigned char  <=     255
3     -32768 <=    short int      <=    32767
4          0 <= unsigned short int <=    65535
5 -2147483648 <=      int         <=  2147483647
6          0 <=    unsigned int   <=  4294967295
7 -2147483648 <=    long int      <=  2147483647
8          0 <= unsigned long int <=  4294967295
```

Ovanstående är en utskrift från [detta program](#).

Nästa lektion

